

1.Descrição cenário e agentes

1.1.Cenário

A nossa implementação trata-se de um motorista de aplicativo genérico (uber, 99, drive, etcetera) onde se consiste na realização de um trajeto, sendo ele, procurar por um passageiro; ir até o passageiro; levar o passageiro até o destino; procurar novamente outro passageiro. Se mantendo nesse loop eternamente. Fica claro que o primeiro agente seria o motorista, mas como a atividade se pede dois agentes pensamentos em adicionar um agente de administração, sendo ele o acaso. Durante o trajeto o motorista pode acabar parando por alguma coisa e é exatamente onde o acaso entra, ela vai determinar o motivo que o motorista parou sendo três acaso possíveis: pedestre atravessando a rua; acidente; ou sinal fechado. Esses serão os nossos agentes definitivos

1.2.Agente: Motorista

O agente motorista se trata do motorista que havíamos falado mais cedo. Ele carrega o papel principal sendo o responsável por grande parte do funcionamento do programa. Ele apresenta três estados: Parking; DrivingToPassenger; DeliveringPassanger. O estado inicial que esse agente se encontra é o “Parking” onde neste estado ele sorteia uma chance de 75% de achar um passageiro. Após achar um passageiro é setado o valor da distância que o passageiro se encontra em relação ao motorista e a distância da viagem em relação ao passageiro em uma variável para cada valor respectivo mencionado aqui.

Agora irei falar sobre o “DrivingToPassenger”. Esse estado faz toda a lógica do trajeto do motorista até o passageiro, avançando um Km por turno e contando 1 minuto por turno também (esses valores são armazenados em uma variável), também é contido a lógica dele parar que é controlada por uma chance de 15% de chance dele parar durante o trajeto, ao parar ele fica imobilizado impedindo que ele avance, só liberado quando o segundo agente, Casualidade, o libere. Ao chegar no final do percurso o passageiro entra no carro e é trocado para o estado “DeliveringPassanger” a lógica do próximo passo é a mesma, por isso prefiro não explicar agora.

1.3.Agente: Casualidades

Esse agente é totalmente dependente para que ele troque de estado, começando no estado “Observando” e trocando para o estado “Evento” que fique nessa vai e vem.

Inicialmente esse agente começa no estado “Observando” onde ele fica monitorando se o motorista parou a cada instância do programa, quando o motorista para ele trocar para o estado “Evento” que somente ali irá liberar o motorista.

Neste estado, “Evento”, lá é sorteado três possíveis casualidades para que o motorista tenha parado, sendo eles: 0. Sinal fechado; 1. Acidente; 2. Pedestre passando. E cada desses eventos ocupam um certo tempo para que libere o motorista para continuar a sua viagem, o impossibilitando de prosseguir e consequentemente o tomando mais tempo. Após o tempo determinado pela casualidade ele aciona um trigger dentro do motorista no qual o permite seguir a viagem.

2. Estrutura e Regras de Implementação

2.1 Regras dos estados

Para a estruturação dos agentes foi usado o mesmo modelo que foi utilizado em sala de aula, onde primeiro era formado uma interface estado que conteria as informações básicas que devem ser usadas como molde, sendo elas o enter, execute e leave. Também foi criado um AbstractState onde ele serviria de base e conteria as informações necessárias para se utilizar e puxar por si mesma e outros scripts.

2.2 Máquina de estados múltiplas e estados

Para que pudéssemos ter mais de um agente presente no código, precisávamos o transformar em um objeto genérico, assim como foi apresentado em sala de aula foi criado uma nova interface, sendo ela a interface Agente, onde estariam métodos pré estabelecidos que devem conter no agente. E para que o estado saiba a qual

agente o pertence, no Abstract State é adicionado um método onde liga o estado ao objeto que lhe foi herdado e passado.

2.3 Objetos

Os objetos eles implementam da interface agente, neles colocamos as variáveis que os estados usaram e os getter, setter, decrease e add para podermos chamar e alterar essas variáveis nos estados. Também não se pode esquecer dos métodos obrigatórios sendo os dois mais importante o update que irá acionar o execute do estado atual. E o setState, responsável por trocar os estados e acionar o leave do estado atual e o enter do próximo estado.

3. Descrevendo o código

3.1. Estrutura dos agentes

Começando pelo agente Motorista. Nele temos as seguintes variáveis(todos possuem um método que puxe eles ou altere os valores deles);

- - String nome_passageiro: Tem como objetivo guardar o nome do passageiro.
- - int KmRodados: Tem como objetivo guardar os kms rodados durante um trajeto.
- - int Kmviagem: Tem como objetivo guardar a distância do trajeto do passageiro até o ponto de chegada dele.
- - int KmPassageiro: Tem como objetivo guardar a distância do motorista até o passageiro.
- - int tempo: Tem como objetivo guardar o tempo durante um trajeto e outro.
- + boolean parado = false: Tem como objetivo determinar se o veículo parou durante o percurso de buscar o passageiro ou entregar ele até o destino. Ele começa em false pois no estado de "Parking" essa variável não é levada em consideração, já que ele está estacionado e não parado. E quando ele entra nos outros estados ele já entra em movimento.
- - State <Motorista> state = new Parking(this): Aqui é criado a variável estado, sendo responsável por definir qual estado atualmente o agente se encontra, além de passar o tipo do agente, sendo ela Motorista.

Agora indo para o agente Casualidades. Nele temos as seguintes variáveis(todos possuem um método que puxe eles ou altere os valores deles);

- - int opcao: É responsável por armazenar o número sorteado entre 0 à 2 para qual evento deverá acontecer.
- - int tempo_parado: É responsável por armazenar a quantidade de tempo que irá ficar parado durante o evento.
- - boolean sorteado = false: É responsável para dizer ao código que o sorteio do evento já aconteceu, isso serve para que o código não sorteie outro evento na próxima interação e resete o código. Ele está em false pois é o estado natural em que se deve estar para que o sorteio aconteça uma primeira vez.
- - boolean tempo_definido = false; É a variável responsável por dizer ao código que o tempo já foi definido, para que assim não resete o timer. Ele está em false pois é o estado natural em que se deve estar para que o tempo não seja sobrepujado.
- - State <Casualidades> state = new Observando(this): A mesma coisa que o agente explicado anteriormente
- - private final Motorista motorista: Aqui é criado uma variável que referência o agente motorista

Aqui temos um caso um método que vale a pena ser comentado, que é o public Casualidades(Motorista motoristas), ao criar o agente de casualidades você deve passar o motorista em sua criação pois ele irá usar esse motorista como referência e mudar os atributos necessários nessa referência.

3.2 Estrutura dos estados

Antes de passarmos estado por estado quero explicar como eu os estruturei durante esse projeto, primeiro que alguns códigos eu os coloquei em função para tornar a legibilidade mais tranquila. Segundo, deixei prints dentro dos estados para explicar o que estava acontecendo, agora partindo para a explicação dos códigos.

3.2.1. Estados do Motorista

Os estados do motorista são no total 3: Parking; DrivingToPassanger; DeliveringPassanger. E são executados nesta ordem que os apresentei agora, os explicando um por um;

Parking

No execute do Parking ele já começa definindo a porcentagem de achar um passageiro sendo dela de 0.75 ou seja de 75%, caso seja verdadeira é criada logo em seguida uma variável temporária com 5 nomes que serve para pegar aleatoriamente com a expressão Math.random que foi usada em todo sorteio. Após isso também é sorteado a distância dos trajetos até o passageiro e o destino usando a expressão Math.random como havia mencionado(nesses dois casos podemos ver que a expressão math.random tem o sinal de *, o motivo disso é para definir o intervalo máximo do sorteio e o + após essa expressão é para definir o intervalo mínimo). E todos esses sorteios são registrados nas variáveis explicadas no capítulo anterior. E após isso é emitido o comando para o próximo estado sendo ela DrivingToPassenger.

DrivingToPassenger/DeliveringPassenger

Irei explicar a parte de logística de como o estado faz para definir se foi dado um km ou não e como para o motorista pára, irei explicar para os dois porque a lógica dos estados é a mesma, só muda uma variável.

Começando pelo execute onde a primeira coisa que ele faz é fazer um sorteio onde tem 85% de o carro realizar seu trajeto normalmente, mas caso isso não aconteça a variável parado é dada como true o que impede de realizar o avanço, pois na verificação tem uma condição extra, sendo ela que a variável parada seja falsa, onde ela só é dada como falsa novamente pelo agente Casualidade.

Agora temos duas variáveis extremamente importantes que é o KmRodados e o Kmpassageiro/KmViagem, onde a cada interação é incrementado +1 em KmRodados e continua assim enquanto o Kmrodados for menor que o KmPassageiro/KmViagem. Além disso também é incrementado +1 na variável Tempo, pois no final do trajeto é mostrado quanto tempo foi gasto para chegar ao destino. Uma coisa interessante é que caso o KmRodados ultrapasse o teto, eu deixo ele igual ao KmPassageiro/KmViagem. E após tudo isso as variáveis Kmrodados e tempo são definidas como 0 e é trocado o estado para o próximo.

3.2.2. Estados do Casualidades

Os estados de Casualidades são no total 2: Observando; Eventos. Diferente do motorista esse agente não segue uma ordem de começo e fim, ele fica nesse vai e vem entre os dois estados, sempre dependendo do que acontece com o motorista. Os explicando um por um;

Observando

Esse é o estado mais simples do projeto todo. Começando pelo execute onde a cada interação ele verifica a variável parado do motorista que foi passado na criação do agente (aquele que expliquei no capítulo 3.1 no public Casualidades(Motorista motoristas)). Caso a variável parado seja false nada acontece e ele continua observando, mas caso ela seja true é emitido a troca para o próximo estado sendo ele Eventos. Ou seja, esse estado serve somente para esperar o motorista parar.

Eventos

Aqui é onde fica toda a lógica das casualidades. Começando pelo execute onde a primeira coisa que ele faz é verificar a variável sorteado, caso ela seja false é feito o sorteio do evento usando a expressão Math.random (aqui o * 3 com o (int) faz com que o resultado seja 0, 1 ou 2) e o resultado é guardado na variável opcao, logo em seguida o sorteado é dado como true para que na próxima interação não seja sorteado outro evento por cima do que já está acontecendo.

Após isso temos um switch onde cada número representa um evento, sendo eles: 0. Pedestre passando; 1. Acidente; 2. Sinal fechado. E em cada um deles é chamada a função definir_tempo que define quanto tempo o motorista vai ficar parado, sendo o pedestre sempre 1 minuto, o acidente entre 2 a 4 minutos e o sinal fechado entre 1 a 3 minutos. Uma coisa interessante é que como o execute é chamado toda interação o definir_tempo também é, por isso existe a variável tempo_definido, onde só é setado o tempo caso ela seja false e logo em seguida ela é dada como true, assim o timer não é resetado a cada interação.

Agora irei explicar as funções dos eventos, irei explicar as três juntas porque a lógica delas é a mesma, só muda a mensagem que aparece. Enquanto o tempo_parado for maior que 0, a cada interação é incrementado

+1 na variável tempo do motorista (pois mesmo parado o tempo da viagem continua passando) e é decrementado -1 no tempo_parado. Quando o tempo_parado chega em 0, as variáveis sorteado e tempo_definido são definidas como false novamente para que o próximo evento possa ser sorteado do zero, a variável parado do motorista é dada como false o que libera ele para continuar o trajeto, e por fim é trocado o estado de volta para o Observando, fechando o ciclo.

3.3 Tabelas

Motorista

Estado	Enter	Execute	Leave	Transição
Parking	Imprime “Procurando passageiros...”	Sorteio de 75% de achar um passageiro. Se achou sorteia um nome aleatório e define as distâncias.		A transição ocorre após a definição das distâncias. Sendo o gatilho principal o sorteio de 75%
DrivingToPassenger		Sorteio de 85% de o motorista seguir viagem, caso não a variável parado é definido como true e bloqueia que a viagem aconteça.	Imprime “Dirigindo até o destino” junto com o KmViagem.	A transição ocorre quando o KmRodados é igual ou superior ao KmPassageiro
DeliveringPassenger		Sorteio de 85% de o motorista seguir viagem, caso não a variável parado é definido como true e bloqueia que a viagem aconteça.		A transição ocorre quando o KmRodados é igual ou superior ao KmViagem

Casualidades

Estado	Enter	Execute	Leave	Transição
Observando		A cada iteração verifica se a variável parado do motorista foi dada como true.		A transição ocorre quando a variável parado do motorista é true, indo para o Eventos.
Eventos		Se ainda não foi sorteado, sorteia um dos três eventos e define o tempo_parado. Enquanto o tempo_parado for maior que 0 é incrementado +1 no tempo do motorista e decrementado -1 no tempo_parado.		A transição ocorre quando o tempo_parado chega em 0, onde o motorista é liberado e é voltado para o Observando.

Estrutura do código

Arquivo	Papel no projeto
Main.java	É a classe que gerencia a execução, onde são criados os agentes e colocados em uma lista, além de rodar o loop principal chamando o update de cada agente a cada 3 segundos.
Motorista.java	Agente A, onde ficam as variáveis do trajeto e a troca de estados do motorista.
Casualidades.java	Agente B, onde ficam as variáveis dos eventos e a referência do motorista que ele observa.
Parking.java DrivingToPassenger.java DeliveringPassenger.java	Estados do agente Motorista.
Observando.java Eventos.java	Estados do agente Casualidades.

4 Saídas esperadas

Prints tiradas diretamente da IDE (VsCode)

```
+-- Motorista@14dad5dc -----
| Estado atual: Parking
| Passageiro encontrado: João, Km da viagem: 4
+-----
+-- Casualidades@4e0e2f2a -----
| Estado atual: Observando
| Observando: Motorista@14dad5dc
+-----

=====

+-- Motorista@14dad5dc -----
| Estado atual: DrivingToPassenger
| Dirigindo até o passageiro: João, Km rodados: 1/4
+-----
+-- Casualidades@4e0e2f2a -----
| Estado atual: Observando
| Observando: Motorista@14dad5dc
+-----

=====

+-- Motorista@14dad5dc -----
| Estado atual: DeliveringPassenger
| Viagem concluída. Passageiro: João, tempo total da viagem: 4 minutos.
| [entrada] Procurando passageiros...
+-----
+-- Casualidades@4e0e2f2a -----
| Estado atual: Observando
| Observando: Motorista@14dad5dc
+-----

=====

+-- Motorista@14dad5dc -----
| Estado atual: Parking
| Passageiro encontrado: Ana, Km da viagem: 2
+-----
+-- Casualidades@4e0e2f2a -----
| Estado atual: Observando
| Observando: Motorista@14dad5dc
+-----

=====

+-- Motorista@14dad5dc -----
| Estado atual: DeliveringPassenger
| Parece que o motorista está preso no trânsito
+-----
+-- Casualidades@4e0e2f2a -----
| Estado atual: Eventos
| Evento: Pedestre atravessando a rua. Motorista parado.
| Tempo restante para voltar a dirigir: 1 minutos.
+-----

=====

+-- Motorista@14dad5dc -----
| Estado atual: DeliveringPassenger
| Parece que o motorista está preso no trânsito
+-----
+-- Casualidades@4e0e2f2a -----
| Estado atual: Eventos
| O motorista já pode voltar a dirigir.
+-----

=====

+-- Motorista@14dad5dc -----
| Estado atual: DeliveringPassenger
| Entregando passageiro: Ana, Km rodados: 6/7
+-----
+-- Casualidades@4e0e2f2a -----
| Estado atual: Observando
| Observando: Motorista@14dad5dc
+-----

=====
```